

Elliptic Curve Diffie-Hellman

Tyler Schill and Megan Judd

What we were trying to do:

For our CS62 final project, we were attempting to implement Elliptic Curve Diffie-Hellman (ECDH) key exchange in C. We came across many useful resources, such as a Python library `ecutils` that does any elliptic curve operations for a given curve. We did not use this library or any online implementation of ECDH and proceeded with our own intuition. Our initial plan was to implement ECDH with no outside assistance - besides OpenSSL - and then see what we could do from there. We were expecting that we would solely have time to implement ECDH itself. Work was delegated quite equally. We tried to split coding time evenly, using the driver-observer two-person coding design. Megan spent more time with the mathematical background and ensured that it was implemented properly. Tyler spent more time on key derivation and AES setup. Regardless, these tasks were not individual. We collaborated thoroughly on each part.

Writing ECDH: Math Matters (Written by Megan)

We began by doing an in-depth read of Aumasson's *Serious Cryptography* chapter 12, which gave good insight into how ECDH works. I was having issues wrapping my head around what an elliptic curve was, how the operation actually works within a group, and further mathematical concepts. Famously, Tyler says to me, "I don't know why we need to go deep into the math behind it; it seems like we have the plug and chug formula right here." I agreed, and we started working together on a toy-curve implementation of ECDH. We successfully got the math

to work in our simple code. Or so we thought. Then, we went on to implement ECDH with the P-256 curve with standardized parameters. This is where we ran into our first real bugs. To successfully find the secret, we needed to consider the G_x and G_y components as a whole. We were initially using G_x and G_y as separate parameters. I built a struct to contain it, and then continued calculating the secret using Bignum mod multiply. We found that this made the code more sensible and improved many of our errors.

Then, we started having segfault errors. Tyler, the master of GDB, spent significant time debugging the code. He found out that point multiplication was causing our segfault error. I turned to *Serious Cryptography* and found two major bug fixes. One, for adding two equal points, there was a separate way to calculate the slope of the connecting line. This edge case cannot be grouped into the regular computation of $P + Q$, where P and Q are distinct. For example, Figure 1 shows the process of calculating $P + Q$ with P and Q distinct, as compared to calculating $P + P$. The main difference is

“The formula to determine the coordinates (x_R, y_R) of $R = P + P$ is slightly different from the formula for a distinct P and Q . The basic formula is $x_R = m^2 - x_P - x_Q$ and $y_R = m(x_P - x_R) - y_P$, but the value of m becomes $(3x_P^2 + a) / 2y_P$, where a is the curve’s parameter, as in $y^2 = x^3 + ax + b$,” (Ausmassan, Ch 12).

Two, we did not consider adding points that are inverses. When points are inverses of each other, P and $-P$, finding their sum does not follow the same process as distinct P and Q . Figure 2 shows how calculating $P + -P$ yields no point of intersection to find the sum, thus giving the point at infinity.

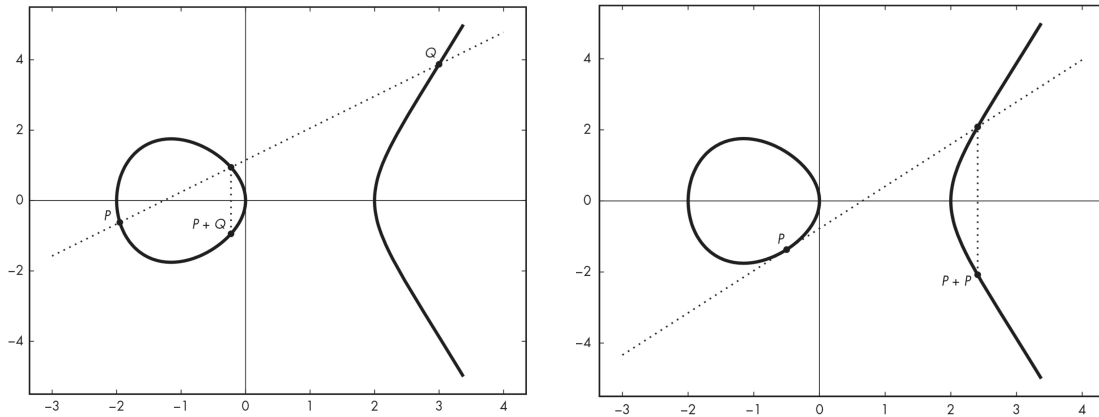


Figure 1: General case for adding distinct points along an elliptic curve (L), versus adding a point to itself (R), *Serious Cryptography*

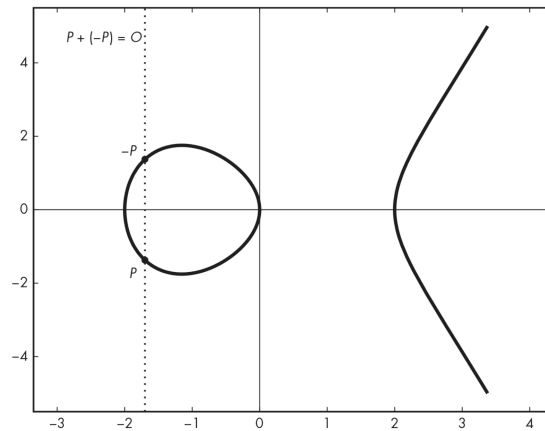


Figure 2: Case for adding inverse points, *Serious Cryptography*

These bugs ultimately required us to delve deeper into how P-256 was mathematically constructed and the general rules of an elliptic curve group. The following day, we worked in the Trust Lab on the point_multiplication function. Our initial assumption, that calculating aB or Ba would be as simple as multiplying the (x,y) components of base G , was incorrect. Multiplication cannot be carried out in this way within an additive group. It requires the composition of addition. Tyler, with his architecture background, explained the efficient way of

multiplying a point by a scalar. He acknowledged in the textbook that if our scalar k is larger, “then computing $k - 1$ additions is downright infeasible,” (Aumassan, ch 12). We implemented scalar multiplication by analyzing the scalar’s bits. If a bit was set, we multiplied the current result by 2 and added the current result to retrieve the updated result. If the bit was not set, we simply multiplied the current result by 2 and stored this updated value. After extensive debugging of this logic and realizing we forgot to free basically every Bignum or Point struct, we finally got our ECDH implementation to work correctly. These were our biggest challenges with coding ECDH, and it affirmed that understanding the math behind these cryptographic frameworks is imperative for implementation. To see our finished product and the way we figured out these mathematical caveats, look at the attached code!

Going further: How do we use ECDH? (Written by Tyler)

While Elliptic Curve Diffie-Hellman (ECDH) was the prime goal, conquering that challenge soon proved unsatisfactory. We had done all this work to compute a shared secret, but what was there to do with it now? We knew that Diffie-Hellman (DH) was typically used in conjunction with asymmetric cryptography schemes, so we set out to learn how this was done in practice. It became clear that the bridge between a symmetric cipher and a Diffie-Hellman shared secret was a key derivation function (KDF).

We had touched on key derivation in class, so at first, I thoughtlessly trusted that this was just the way that DH worked alongside a symmetric cipher. However, I started wondering why this middleman was necessary. It seemed that we had generated a secret that would be hard for an adversary to guess. While calculating the shared secret when it can be an arbitrary length is difficult, calculating it from a fixed bit length is less so. For instance, take a secret containing

256 bits that is a candidate to be used with AES-256 encryption. The 256-bit number is only ever going to be the x coordinate of a point on the curve. Not every 256-bit number will be on the curve, meaning some bits may not ever get set, or they get set following a pattern (pattern recognition benefits adversaries!). Another downside of using the secret is its arbitrary length. While one could just take the first 256 bits of the secret and use it as the key, a key derivation function is more practical in keeping the security of a large shared secret while also fitting it within the specified key length.

Now I was convinced of key derivation's merit, but it still seemed like the underlying logic could be a little scary. After consulting IETF's RFC pages, though, it was clear that this was not the case. The basic premise followed that you could hash the secret data in order to create a key of the length that a symmetric encryption algorithm expected. Based on everything we had discussed in class about hashing, this was very intuitive. We were binding data to some value of a fixed length. Additionally, the RFC mentioned adding a random "salt" value that would protect the hash against a rainbow table attack.

Given time constraints, we made the implementation choice of only hashing the shared secret, and using this value as the key. The reasoning behind this was that the main concerns with the shared secret were its non-uniform bit distribution and its length. Both of these aspects were addressed with just the hash. The salt and RFC's recommendation of a hashed key derivation (based on HMAC) would be nice to explore, but were not a necessity.

One last note on key derivation is that Jonathan brought up PBKDF, or password-based key derivation functions. Before committing to trying to implement a key derivation function, we had used OpenSSL's PBKDF function and it had worked. However, upon further research on the difference between PBKDFs and normal KDFs, I found that the former purposefully takes a

longer time to compute a key (possibly by hashing a large number of times). This is in an attempt to make an adversary have to perform an infeasible amount of computation to reveal the password. Given that the shared secret already had a secure amount of entropy, this would be redundant and thus unnecessary.

Once we had produced a key, we wanted to use it with a symmetric encryption algorithm. Though possibly a worthy challenge for a longer term, we thought that implementing a symmetric cipher would be too much work for this project. Thus, we outsourced this part to the OpenSSL library, using its symmetric encryption interface with the AES-256-CBC algorithm. Now, our project had come to full fruition, as we were able to go from a shared secret produced by Diffie Hellman, to the practical ability to encrypt/decrypt any data we wanted.

Takeaways:

As mentioned, our main goal was to wrangle with the math behind elliptic curves and apply it to a cryptographic scheme that isn't always paired with it (we were first introduced to Diffie Hellman based on finite fields, not elliptic curves). The implementation made us question our intuition on even simple mathematical operations like addition and multiplication, as dealing with points on an elliptic curve introduces new schemes for these basic principles. However, despite the abrasion of coding it up, we were able to recognize how these operations were the heart of the elliptic curve's security.

Another key idea we saw play out was entropy/randomness. At multiple points of our computations there were added steps that injected randomness into the operations. For example, we used ephemeral values for Alice's and Bob's secrets within the DH key exchange. This ensured forward-secrecy. Additionally, the salt included in the key derivation function was not a

necessity in our small scale example, but in the real-world provides even more security against attacks. While utilizing this randomness did not require any extra hard math, we saw how much it was able to affect the security of our program.

Ultimately, we were able to achieve our goal with this project, and even extend it a little bit to explore other related topics more in depth. Also, we were able to get some extra practice with the OpenSSL library, especially with the addition of implementing AES encryption/decryption. While tedious, this is surely not a bad thing! Given the prevalence of elliptic curves (and Diffie Hellman) within modern applications, this project will hopefully send us out into the world with a better understanding of the systems around us.

Sources:

J.-P. Aumasson. *Serious Cryptography: A Practical Introduction to Modern Encryption*. 2nd edition. No Starch Press, 2024

Keith Martin. *Everyday Cryptography: Fundamental Principles and Applications*. 3rd Edition, 2025.

Internet Engineering Task Force. (2011). *US Secure Hash Algorithms (SHA and SHA-Based HMAC and HKDF) (RFC 6234)*.

National Institute of Standards and Technology. (2023). *Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters (NIST SP 800-186)*.